Clase 2 · Permisos UGO, bit SetUID, /etc/passwd y fork()

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 27 ago 2026 · Tema 1 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Permisos UGO y chmod

"ugo" (Hugo sin H): **u**suario (dueño), **g**rupo, **o**tros (no "owner"). chmod = change mode.

Permiso	Valor
r	4
w	2
x	1

- `chmod 777`: todo para todos. `chmod 644`: dueño rw, resto r. `chmod 755`: ejecutable típico (rwx, r-x, r-x).
- Para ejecutar un binario también hace falta el permiso de lectura.

2. Bit SetUID

Quien ejecuta toma temporalmente los privilegios del **dueño del archivo** mientras dura la ejecución. Cuarto dígito a la izquierda:

Dígito	Bit
4	SetUID (usuario), el más usado
2	SetGID (grupo)
1	Sticky (otros)

Ej.: `chmod 4511` → `r-s--x--x` (la **s** en lugar de la x). **Riesgo:** un ejecutable de root con SetUID convierte en root a cualquiera durante esa ejecución. No es un cambio permanente de la cuenta.

3. /etc/passwd, ID real y efectivo

- **ID real:** dueño legítimo, asignado en `/etc/passwd`; `getuid()`.
- **ID efectivo:** con qué privilegios corre el programa en ese instante; `geteuid()` (e = effective).
- Campos de `/etc/passwd`: `usuario : x : UID : GID : nombre real (GECOS) : /home/… : shell`. El primero es root; la `x` = contraseña oculta; shell `nologin` impide sesión interactiva.

4. Tarea 2: probar SetUID

1. Como usuario normal: `fdisk -l` → permiso denegado.

2. Como root: `ls -l /sbin/fdisk` → 755.
3. Como root: `chmod 4511 /sbin/fdisk` → `r-s--x--x`.
4. Como usuario normal: `fdisk -l` ya funciona.
5. **Revertir de inmediato:** `chmod 755 /sbin/fdisk`.

- Actividad 2: `programa02_ids.c` imprime `getuid()` y `geteuid()`; con `chmod 4511` y ejecutado por otro usuario salen distintos.

> **Para sacar 10**
>
> Las capturas deben mostrar **ID real ≠ ID efectivo**. Error común: probar desde root (ambos salen 0). En esta clase se dijo no entregar todavía (sin demo en el lab).

5. fork()

- Llamada del kernel que **clona** toda la memoria del proceso (costoso). El hijo es copia idéntica salvo **PID y PPID**. Corren en **paralelo y asíncronos**.

> **Retorno (examen)**
>
> `fork()` devuelve `pid_t`: **al padre, el PID del hijo**; **al hijo, 0**. Se separa con `if (id_hijo != 0) { padre } else { hijo }`. Sin `if`, todo lo de después se ejecuta dos veces.
>
> - Antes del fork es el "proceso main"; "padre" solo cuando ya hay hijo.
>
> | Dato (programa05_fork_v2.c) | Valor |
> |---|---|
> | PID del padre | 30405 |
> | PID del hijo | 30406 |
> | `id_hijo` en el padre / en el hijo | 30406 / 0 |
> | `getppid()` del padre | PID de la shell (bash) |
> | `getppid()` del hijo | 30405 |
>
> - No se puede predecir quién imprime primero (planificador).
>
> > **Deadlock (examen)**
> >
> > Si el padre espera al hijo y el hijo espera al padre → bloqueo indefinido; solo sale con Ctrl+C.

Pendiente

Tarea moral: leer el PDF 2 del Tema 1 (procesos).

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
UGO	Usuario, grupo, otros; r 4, w 2, x 1
SetUID	4.º dígito 4; se ve `s`; privilegios del dueño solo durante la ejecución
Real / efectivo	`getuid()` / `geteuid()`
fork	Padre recibe PID del hijo; hijo recibe 0; copia salvo PID/PPID
Deadlock	Padre e hijo se esperan mutuamente